



Top 10 Python “Loop” Tricky Interview Questions

⚠️ THESE ARE NOT TRICK
PUZZLES, THEY ARE
REAL BUGS FOUND IN
PRODUCTION PYTHON
SYSTEMS.

1. Self-Extending Loop

```
nums = [1, 2, 3]
for n in nums:
    if n == 2:
        nums.append(4)
    print(n)
```

✓ Output

1
2
3
4

🧠 Explanation

The iterator sees newly appended items.
Loop length changes during iteration.

2. The “Looks Finite” Infinite Loop

```
i = 1
while i < 5:
    print(i)
    i = i / 2
```

✗ Output

Infinite loop printing:

1
0.5
0.25
0.125

...

🧠 Explanation

i never reaches $\geq 5 \rightarrow$ infinite fractional decay.

3. for-else + continue trap

```
for i in range(3):  
    if i == 1:  
        continue  
    if i == 2:  
        break  
else:  
    print("Completed")
```

✅ Output
(no output)

🧠 Explanation

break prevents else even if continue was used earlier.

4. Range Snapshot Bug

```
nums = [1, 2, 3, 4]  
for i in range(len(nums)):  
    nums.pop()  
print(nums)
```

✅ Output
[]

🧠 Explanation

range(len(nums)) is evaluated once → 4 iterations no matter how list shrinks.

5. Mutating Dict During Loop

```
d = {"a":1, "b":2, "c":3}
for k in d:
    if k == "b":
        d.pop(k)
print(d)
```

✗ Output

RuntimeError: dictionary changed size during iteration

6. Generator Exhaustion Trap

```
gen = (i for i in range(3))
for x in gen:
    print(x)
for x in gen:
    print(x)
```

✓ Output

0

1

2

🧠 Explanation

Generators are one-time iterables.

7. Shadowing Loop Variable

```
i = 5  
for i in range(3):  
    pass  
print(i)
```

✓ Output

2

🧠 Explanation

Loop overwrites outer variable.

8. Chained While Logic

```
i = 10  
while i:  
    i -= 3  
print(i)
```

✗ Output

Infinite loop:

7

4

1

-2-5

...

🧠 Explanation

Negative numbers are still True.

9. Loop That Never Ends (Looks Safe)

```
i = 0
while i != 1:
    i += 0.1
```

✗ Output
Infinite loop.

🧠 Explanation

Floating-point precision — i never equals exactly 1.

10. Iterator vs List Mutation

```
nums = [1,2,3]
it = iter(nums)
nums.append(4)
print(list(it))
```

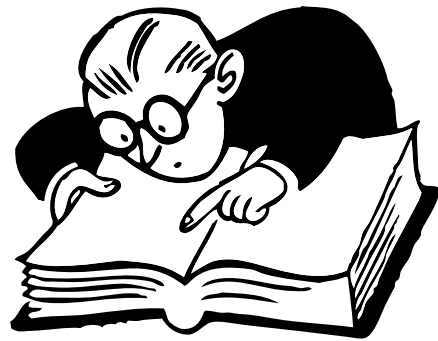
✓ Output
[1, 2, 3, 4]

🧠 Explanation

List iterator reflects runtime mutations.
iter(nums) does not copy the list.

So now you have:

```
nums —————▶ [1, 2, 3, 4]
it   —————▶ same object, same
memory
```



Comment Questions



Thank You